

PRODUCT REQUIREMENTS DOCUMENT

MemStack Desktop Agent Client

统一通用 Agent 与编程 Agent 的桌面指挥中心

Version: v1.1 Draft

Date: 2026-07-10

Target: macOS / Windows desktop

Status: Product structure and visual direction locked

Scope: Product PRD + UI/UX specification + selected prototype

核心决策：不建设两套彼此割裂的产品。Work 与 Code 共享项目、任务、线程、记忆、权限、运行状态和产物，差异只体现在任务能力集与自适应工作画布。

Research and product framing

The research brief below summarizes current GitHub Copilot App and Codex / ChatGPT Desktop patterns, public UX failure signals, and the design implications for MemStack. Full source notes remain in the companion Markdown report.

研究对象：GitHub Copilot App、Codex / ChatGPT Desktop、MemStack 当前桌面实现 目标用户：知识工作者、产品/运营人员、软件工程师、技术负责人、企业管理员 时间范围：重点覆盖 2026 年 2 月至 2026 年 7 月的公开资料 研究日期：2026-07-10

1. Executive Read

Agent 桌面产品的核心竞争已经从“模型能否完成任务”转向“用户能否同时指挥、观察、审阅和接管多个长任务”。GitHub Copilot App 的优势是把 Issue、会话、Worktree、PR、CI 与合并闭环放进同一产品；Codex / ChatGPT Desktop 的优势是把通用知识工作、文件产物、浏览器、计算机操作和本地编程环境放入同一任务系统。两者共同证明，聊天不是最终的信息架构：聊天适合表达意图和消除歧义，真正的工作需要可检查的画布、产物和决策点。

公开用户反馈集中在四类高风险问题：运行状态不可相信、Worktree/分支语义不清、长任务恢复与历史列表不可靠、客户端在大型工作区或多任务下性能下降。对 MemStack 来说，机会不是简单复制两者，而是把企业记忆和可审计决策链作为共享内核，让通用工作与编程任务在一个任务对象中连续流转。

结论：MemStack 应采用“统一任务内核 + 双能力模式 + 自适应工作画布”。主导航围绕 Home、My Work、Automations、Search 和 Projects；任务页围绕状态、计划、产物、环境、审批和可恢复性，而不是围绕聊天消息数量。

2. 竞品产品模型

2.1 GitHub Copilot App

观察证据：

- GitHub 将 Copilot App 定义为面向 Agent 驱动开发的桌面应用，核心能力包含并行工作区、GitHub 原生 Issue/PR 上下文、会话模式、自动化、Quick chats 和 Canvas。 [GitHub Copilot App 概念文档](#)

- 每个会话可运行在新 Worktree、本地仓库或云沙箱中，并提供 Interactive、Plan、Autopilot 三种自主程度。[Agent sessions 文档](#)
- My Work 将 Issue、PR 和活跃工作集中展示，Issue 可以预加载上下文并默认进入 Plan；PR 可查看概览、CI、Review activity、Files changed，并发起修复或 Agent Merge。[Issue 与 PR 工作流](#)
- Canvas 被定位为“人与 Agent 共同操作的工作表面”，可承载计划、浏览器、终端、部署、仪表盘和流程状态。[Canvas 文档](#)
- Automations 支持本地或云端运行、手动/定时/Issue 触发，并在创建时限定可用工具。[Automations 文档](#)

设计优点：

- 用 My Work 解决“我现在应该处理什么”，而不是把用户直接扔进空聊天。
- GitHub 对象天然成为上下文和交付物，Issue 到 PR 的闭环非常短。
- Quick chat 与正式 Session 分开，降低无意义 Worktree 的创建成本。
- Canvas 明确承认聊天不足以承载复杂工作的事实。

局限与推断：

- 产品价值高度绑定 GitHub 对象模型；跨文档、数据、内部业务系统的通用工作不是主路径。
- 2026 年 6 月后公开上线时间较短，社区问题样本仍偏少；当前不能从少量讨论推断普遍满意度。
- 功能持续扩张后，My Work、Sessions、Automations、Canvas、PR Review 之间可能出现入口竞争，需要严格的任务模型约束。

2.2 Codex / ChatGPT Desktop

观察证据：

- Codex App 最初被定义为“Agent 的指挥中心”，强调项目分组线程、并行 Agent、Worktree、Diff 审阅、Skills 与 Automations。[Codex App 发布说明](#)

- 2026 年 4 月更新将能力扩展到计算机操作、插件、图像、PR 审阅、多文件、多终端、SSH Devbox 和内置浏览器，方向从纯代码工具走向通用工作平台。[Codex for \(almost\) everything](#)
- 当前桌面产品将 Project、Chat、Work 和 Codex 放入同一应用。Project 统一组织聊天、任务、文件和来源；Chat / Work 改变处理方式，而不是改变会话归属。[Projects, chats, and tasks](#)
- Codex 任务可选择 Local、Worktree 或 Cloud；Worktree 支持并行任务和 Local/Worktree Handoff。[环境模式](#)、[Worktree 文档](#)
- 每个任务有项目/Worktree 作用域终端，Agent 可读取当前终端输出；审阅面板支持行级反馈、Stage、Revert、Commit、Push。[集成终端](#)、[Code review](#)
- 内置浏览器为人与 Agent 提供共享网页和本地应用视图，但与日常浏览器 Profile 隔离。[Browser 文档](#)

设计优点：

- 通用工作、文件产物与编程环境共享项目和任务概念，跨场景连续性强。
- Local / Worktree / Cloud 是任务启动时的一等选择。
- Terminal、Browser、Review pane 均是工作表面，而不是日志附件。
- Skills / Plugins 让产品能扩展到企业工作流，而不把全部能力硬编码在主界面。

局限与公开反馈：

- 有用户报告活跃任务实际仍在运行，但 UI 停留在 Thinking、Stop 无效或重启后任务不可见；这直接伤害“Agent 指挥中心”的安全基础。[openai/codex#24287](#)
- Worktree 入口、Detached HEAD 下可用操作和 Diff 范围曾产生明显困惑。[openai/codex#11181](#)、[openai/codex#10704](#)
- 大型工作区扫描可能引发卡顿或崩溃，用户需要可见的索引范围和排除规则。[openai/codex#10996](#)
- 本地数据库仍有会话但侧栏未显示的反馈，说明历史与恢复必须作为产品状态而不是仅作为列表渲染。[openai/codex#21076](#)

- Reddit 上关于并行 Worktree 的反馈同时包含高价值与高困惑信号，适合用来发现问题，不足以估计发生率。[规模化使用讨论](#)

3. MemStack 当前桌面实现

仓库证据：

- 已存在 Tauri + React 桌面应用：[agi-stack/apps/desktop](#)。
- 已实现 Session 侧栏、Runs、Agents、Memory、Artifacts、Runtime、Chat、Board、Workspace Dock、Terminal、Changes、Pull request、Plan、Tool events 和 Artifacts 等大量表面。
- 当前视觉采用深色基础色 #080C12，面板 #0F141D / #151A24，状态色 Cyan/Green/Amber/Red，适合延续。
- 当前 App 主函数覆盖约 3,000 行并直接协调大量状态和表面。后续大规模 UX 实现前，应将 Shell、Task State、Navigation、Workspace Canvas 和 Approval 分离，否则设计迭代成本会持续上升。

当前体验问题（基于本地实现与页面检查）：

- Quick links 同时包含 Runs、Agents、Memory、Artifacts、Runtime，而 Session/Workspace 又拥有 Changes、PR、Plan、Terminal、Tool events、Artifacts，存在全局对象与任务内对象的层级重复。
- 登录前主界面已经暴露大量编程概念，通用用户难以理解第一步。
- “Runs、Sessions、Workspaces、Conversations、Tasks”在界面和后端中同时存在，需要统一用户语言。
- 右侧 Workspace Dock 已具备演进为自适应画布的基础，但当前更像多 Tab 工具箱，尚未围绕当前决策组织。

4. 排名后的 UX 问题

P0：运行状态不可信

- 用户目标：明确知道 Agent 是否仍在运行、等待、暂停、断开、失败或已经完成。

- 破坏点：UI 与真实执行脱节时，用户可能重复发起任务、无法停止副作用，或误以为工作丢失。
- 严重度：Critical。
- 频率信号：多个公开 Issue；无法从公开样本量估计整体比例。
- 置信度：高（问题存在），中（普遍性）。
- 产品动作：服务端权威状态 + monotonic revision；每个任务显示最后心跳、环境、执行 ID、可恢复动作；UI 不得用模糊 “Thinking” 覆盖所有状态。

P0：环境、Worktree 与分支语义不清

- 用户目标：知道 Agent 在哪里改了什么，以及这些改动如何进入本地或 PR。
- 破坏点：入口消失、Detached HEAD、Local/Worktree Handoff 和 Diff 范围不一致会让用户无法预测结果。
- 严重度：High。
- 频率信号：多个 Codex Issue 与社区讨论。
- 置信度：高。
- 产品动作：任务 Header 永久显示 Environment + Checkout + Branch；危险/不可用动作解释原因；Handoff 使用可预览步骤而不是单一动词。

P0：审批脱离工作上下文

- 用户目标：在看到风险、变更、来源和影响范围的同一处做决定。
- 破坏点：如果审批只出现在聊天卡片或全局通知，用户需要在多个表面拼接证据。
- 严重度：High。
- 频率信号：GitHub 对审阅负担的官方问题陈述，且多 Agent 输出会放大该问题。[GitHub Copilot App 发布文章](#)
- 置信度：高。
- 产品动作：审批卡固定在受影响的 Canvas 旁边；显示动作、目标、数据、风险、可逆性与适用范围。

P1：聊天滚动无法承载真实工作

- 用户目标：直接检查计划、文档、表格、网页、Diff、终端和最终产物。
- 破坏点：日志和工具调用淹没意图，最终产物难以发现或继续编辑。
- 严重度：High。
- 频率信号：GitHub Canvas 与 Codex 多工作表面方向一致。
- 置信度：高。
- 产品动作：Canvas 为主、Timeline 为辅；工具活动默认折叠；产物有版本和生命周期。

P1：任务队列不能表达“现在需要我做什么”

- 用户目标：在多个项目和 Agent 中迅速找到 Needs input、Needs approval、Ready to review。
- 破坏点：仅按最近时间列 Session，会把重要决策淹没在活跃日志中。
- 严重度：Medium-High。
- 频率信号：GitHub My Work 的产品结构直接回应此问题。
- 置信度：高。
- 产品动作：My Work 使用语义分组，不默认按聊天最近活动排序。

P1：长任务恢复与历史缺少可验证性

- 用户目标：重启、断网、跨设备或切换项目后继续工作。
- 破坏点：侧栏丢失条目、缓存与服务端状态不一致、重新发送造成重复执行。
- 严重度：High。
- 频率信号：公开 Issue 与多窗口反馈。
- 置信度：中高。
- 产品动作：独立的恢复页、重复执行保护、运行 revision、最近同步时间、显式“Reconnect / Resume / Fork recovery”。

P2：大型项目扫描与多任务性能缺少边界

- 用户目标：知道索引什么、为什么变慢、如何排除大目录。

- 破坏点：应用卡顿或崩溃，但用户只能通过移走目录规避。
- 严重度：Medium-High。
- 频率信号：少量但高影响的公开 Issue。
- 置信度：中。
- 产品动作：索引状态与排除规则可见；默认尊重 `.gitignore`；大目录预警；后台索引资源预算。

5. 机会地图

本周可做

- 统一状态词：Running、Paused、Needs input、Needs approval、Ready to review、Failed、Disconnected。
- 在任务 Header 永久显示 Environment、Branch、Execution ID 和 Last update。
- 将通用任务与代码任务放进同一 My Work 语义队列。
- 将 Tool events 默认折叠，保留当前动作和验证结果。

本季度可做

- 自适应 Canvas：Work 模式的 Sources/Browser/Document/Data/Artifact；Code 模式的 Plan/Browser/Changes/Terminal/Artifact。
- 上下文审批：每个审批带风险、范围、可逆性和证据。
- 任务恢复协议与跨窗口唯一运行状态。
- 统一 Project / Task / Thread / Run / Artifact 用户模型。

需要进一步验证

- 非开发者是否理解 Work / Code，或更适合“能力标签 + 自动建议”。
- 多任务用户更偏好 My Work 队列还是项目树作为默认首页。
- 右侧审批 Rail 与 Canvas 内嵌审批的可发现性差异。
- 企业管理员对本地、云端和连接器数据边界的理解成本。

6. 产品差异化

MemStack 不应定位为 GitHub Copilot App 的替代 IDE，也不应只做 Codex App 的外观复制。其差异化是：

- 企业记忆原生：来源、历史决策、实体关系和组织知识可追溯。
- 一项任务可跨 Work 与 Code，而不用复制上下文或新开产品。
- 每个主观决策由 Agent 生成结构化理由并进入审计链。
- 本地、Worktree、Sandbox、Cloud 和企业连接器共享一套权限模型。
- 最终交付以可审阅 Artifact 为中心，聊天只是协作过程的一部分。

7. Source Map

高信号官方来源：GitHub Docs / Blog、OpenAI 发布说明、ChatGPT Learn Codex 文档。问题发现来源：openai/codex Issues、Reddit。社区来源只用于发现问题与表述方式，不用于估算发生率。本地来源：[agi-stack/apps/desktop](#) 代码、现有 QA 记录、运行中的本地页面与设计原型。

Product requirements

版本：v1.1 Draft 日期：2026-07-10 状态：产品结构与视觉方向已锁定 (My Work Mission Control) 目标平台：macOS、Windows；Linux 进入 P1 评估

1. 产品摘要

MemStack Desktop 是面向企业知识工作与软件工程的 Agent 指挥中心。用户在同一项目内创建任务，选择或切换 Work / Code 能力模式，监督 Agent 的计划、执行、验证和交付，并在上下文中完成授权、审批、接管和恢复。Workspace 管理员还能在同一治理工作台中配置 Models、Skills、Plugins 和 Agents，明确它们的版本、作用域、依赖、权限和使用关系。

产品的最小可用承诺不是“能聊天”，而是：

1. 用户始终知道 Agent 正在做什么、在哪里做、是否仍在运行。
2. 通用工作和编程工作共享上下文、记忆、权限、状态和产物。
3. 任何需要人类判断的动作都在相关证据旁边完成。
4. 最终成果以可检查、可版本化、可继续编辑的 Artifact 交付。
5. 应用重启、网络中断或窗口切换后，任务状态可恢复且不会重复执行。

2. 背景与问题

当前 Agent 产品常把复杂工作压进聊天滚动流。用户必须在消息、日志、终端、Diff、浏览器、文件和 PR 之间反复切换，难以回答：

- 哪些工作正在运行？
- 哪些任务正在等我？
- Agent 使用了哪些来源和权限？
- 变更发生在本地、Worktree 还是云端？
- 我批准的具体是什么，影响范围多大？
- 应用重启后是否仍是同一个执行？

MemStack 现有桌面应用已经拥有多数底层表面，但入口和对象层级较多，通用用户与开发者缺少统一的产品模型。

3. 目标与非目标

3.1 产品目标

- 建立一套支持 Work / Code 的统一桌面信息架构。
- 将任务状态、环境、产物、审批和恢复做成一等能力。
- 支持用户并行监督多个长任务，而不丢失上下文。
- 在不替代完整 IDE 的前提下完成核心开发闭环。
- 用企业记忆和结构化决策链形成可审计差异化。
- 将模型、技能、插件与智能体从分散设置提升为可搜索、可版本化、可审计的 Workspace 资源。

3.2 非目标

- 不实现完整源代码编辑器、Git GUI 或通用浏览器替代品。
- 不在 MVP 中覆盖所有第三方 SaaS；先建立连接器和权限框架。
- 不展示或持久化模型隐藏推理；只展示可审计计划、动作、证据和结果。
- 不以“更多面板/更多指标”为成功标准。
- 不允许 Work / Code 各自形成独立的项目、通知或历史系统。

4. 目标用户

Persona A：知识工作者 / 产品负责人

- 需要研究、分析、文档、表格、网页和内部系统协作。
- 不理解 Worktree、Commit 等概念，也不应被迫理解。
- 关心来源、交付物、隐私边界、审批和可分享结果。

Persona B：软件工程师

- 需要仓库上下文、计划、终端、Diff、测试、分支和 PR。
- 关心 Agent 在哪个环境、哪些命令被执行、验证是否可信。
- 需要在并行任务之间快速切换并安全合并结果。

Persona C：技术负责人 / Reviewer

- 同时监督多个 Agent 和项目。
- 关心 Needs approval、Ready to review、风险、测试、CI 和变更范围。
- 需要快速决定：批准、要求修改、暂停、转交或合并。

Persona D：企业管理员

- 配置模型、连接器、权限、网络、数据保留、审计与预算。
- 关心本地/云端数据边界和外部写操作。

5. 核心 Jobs To Be Done

1. 当我有一个模糊问题时，我可以先快速讨论，不创建正式执行环境。
2. 当任务需要持续工作时，我可以把它放进项目并选择 Work / Code。
3. 当 Agent 在后台运行时，我可以确认状态、进度、来源、成本和下一决策。
4. 当 Agent 需要授权时，我可以看到动作、目标、数据、风险和可逆性后再批准。
5. 当通用任务产生代码需求时，我可以切换到 Code 而不复制上下文。
6. 当代码任务需要研究、文档或业务系统时，我可以不使用 Work 能力而不离开任务。
7. 当工作完成时，我可以审阅 Artifact、Diff 或 PR，并要求修改或交付。
8. 当客户端中断时，我可以恢复同一个执行而不是重复运行。

6. 产品对象模型

```
[mermaid]
flowchart LR
    U[User / Team] --> P[Project]
    P --> T[Task]
    T --> TH[Thread]
    T --> R[Run]
    T --> C[Context]
    T --> A[Artifacts]
    T --> D[Decisions]
    T --> E[Environment]
    R --> W[Work capability]
    R --> K[Code capability]
    W --> A
    K --> A
    D --> R
```

定义：

- Project：长期上下文容器，包含指令、成员、来源、记忆、仓库和策略。
- Task：用户要完成的结果，是导航、状态和通知的核心对象。
- Thread：围绕任务的对话与协作历史，不等同于执行状态。
- Run：一次可恢复的 Agent 执行，拥有唯一 ID、revision、状态、预算和心跳。
- Context：文件、连接器、网页、Issue、记忆、仓库和用户提供的信息。
- Artifact：文档、表格、图像、网页、文件、Diff、终端会话、PR 等可检查输出。
- Decision：批准、拒绝、要求修改、选择方案、授予权限等结构化人类判断。
- Environment：Work 的数据作用域或 Code 的 Local / Worktree / Cloud 执行位置。
- Model：可路由的推理资源，包含 Provider、Model ID、上下文、预算、超时、健康状态和 Fallback。
- Skill：可复用的 Agent 能力契约，包含 Instructions、输入输出、允许工具、验证与版本。
- Plugin：连接外部系统的能力包，包含权限、工具、安装版本、连接状态和审计事件。
- Agent：可执行任务的角色配置，由 Model、Skills、Plugins、Memory scope、Autonomy 和 Permission policy 组装。

7. 信息架构

7.1 全局导航

- Home：今日摘要、最近项目、恢复任务、建议动作。
- My Work：跨项目语义队列。
- Manage：Models、Skills、Plugins、Agents 的统一资源目录与治理工作台。
- Automations：定时、事件触发和手动工作流。
- Search：跨 Project / Task / Thread / Memory / Artifact 搜索。
- Projects：项目树和项目内任务。
- Notifications：只承载需要注意的状态变化，不复制 My Work。

- Settings : 个人偏好、浏览器、终端、外观、通知与组织级策略入口；高频资源管理进入 Manage。

7.2 My Work 分组

默认只显示：

1. Needs your input
2. Needs approval
3. Running
4. Ready to review
5. Scheduled / Recent (折叠)

禁止仅用“最近活动时间”作为默认排序。每个条目至少显示 Task、Project、Mode、真实状态、最后更新和需要的动作。

7.3 任务工作区

- Header : Task、Project、Mode、Environment、Branch/Scope、Run state、Elapsed、Usage、Pause/Stop。
- Timeline : 用户意图、计划、关键动作、验证、决策；工具细节默认折叠。
- Canvas : 当前工作表面，根据 Mode 和 Artifact 自适应。
- Decision rail : 当前唯一需要用户处理的决策；无决策时折叠。
- Command deck : Steer、补充上下文、切换 Mode、模型、自主程度。

7.4 Manage 统一资源工作台

Manage 使用“资源类型 → 目录 → 详情/配置”的稳定三栏结构。Models、Skills、Plugins、Agents 共享搜索、状态、作用域、Owner、版本和审计语言，不在全局导航中拆成四个互相独立的产品。

- Resource rail : Models、Skills、Plugins、Agents 及数量、说明、当前类型。
- Catalog : 搜索、Active/Attention 筛选、状态、版本、简述和选中态。
- Detail : 身份摘要、作用域、Owner/Provider、状态、专用 Tab 与主动作。

- Relationship graph : 展示“哪些 Agent 使用此 Model/Skill/Plugin”以及 Agent 的模型、技能、插件依赖。
- Governance : 所有编辑生成版本和审计记录 ; 安装 Plugin 不自动扩张任何 Agent 权限。
- Creation : 新建 Skill/Agent 先生成 governed draft , 完成依赖、权限和验证后才能 Publish/Enable。

8. 双模式定义

8.1 Work (通用 Agent)

默认 Canvas : Sources、Browser、Document、Data、Artifact。

核心能力 :

- 深度研究与可追溯来源。
- 文档、表格、演示、图像和结构化报告。
- 浏览器与桌面应用协作。
- 企业连接器读写 (受权限控制) 。
- 项目记忆、组织知识与历史决策检索。
- 自动化与长期任务。

8.2 Code (编程 Agent)

默认 Canvas : Plan、Browser、Changes、Terminal、Artifact。

核心能力 :

- 仓库理解、Issue/PR 上下文。
- Local / Worktree / Cloud 环境。
- 文件变更、Diff、行级反馈。
- 集成终端、测试与本地预览。
- Stage、Revert、Commit、Push、Create PR、Handoff。
- 多 Agent 并行与隔离。

8.3 切换原则

- Mode 是任务能力集，不是新的产品或新的历史列表。
- 切换前展示将新增的工具与权限，不重建 Task。
- Code → Work 保留 Environment 和仓库上下文；Work → Code 必须选择仓库与执行环境。
- 切换不会自动授予更高权限。

9. 核心用户流程

9.1 Quick chat → 正式任务

1. 用户从 Home 创建 Quick chat。
2. 系统仅进行讨论和只读上下文检索。
3. 当出现持续执行需求，系统建议“Create task”。
4. 用户选择 Project、Work / Code、Scope/Environment、Autonomy。
5. 新 Task 继承必要的聊天摘要和显式选择的上下文。

9.2 通用工作任务

1. 用户在 Project 内创建 Work Task，填写 Goal 并选择 Sources、文件或连接器。
2. 系统进入 **Plan-only** 状态；Agent 可以理解目标和读取获准上下文，但不能运行工具或产生外部副作用。
3. Agent 通过结构化 Plan tool-call 生成步骤、每步产物、预计耗时、上下文范围与审批边界。
4. 用户在 Plan Review 中编辑、禁用、增补步骤，或要求 Agent 重新规划。
5. 用户点击 Approve & start 后，系统记录计划版本与授权快照并创建 Run；未批准不得进入 Running。
6. Timeline 显示阶段进度，Canvas 显示真实工作产物；外部写操作仍触发上下文审批。
7. 完成后进入 Ready to review，主 CTA 为 Review artifact；用户可批准、要求修改、导出或分享。

9.3 编程任务

1. 用户填写代码目标，选择仓库和 Local / Worktree / Cloud。
2. Agent 在只读阶段生成可编辑 Plan，包含影响分析、回归测试、实现、验证与 Review packet。
3. Plan Review 固定展示预计耗时、Usage、Environment、上下文与“Ask before external side effects”等边界。
4. 用户批准后才创建 Worktree/Checkout；Header 固定显示 Environment、Checkout/Worktree、Branch。
5. Agent 按批准计划编辑、运行测试、启动 Preview；Changes 与 Browser/Terminal 在同一 Canvas 中切换或分屏。
6. 高风险变更在 Diff 旁触发审批；用户 Stage / Commit / Push / Create PR，或 Handoff 到 Local。
7. CI/Review 结果回流同一 Task，完成后归档。

9.4 任务恢复

1. 应用启动后从服务端/本地权威存储读取 Run revision。
2. UI 显示 Reconnecting，而不是假定 Failed。
3. 若发现活跃执行，提供 Reattach；若无法确认，提供 Inspect status。
4. Resume 必须使用原 Run ID；Fork recovery 才创建新 Run。
5. 任何重复发送都进行 idempotency 检查。

9.5 Workspace 资源管理

1. 用户进入 Manage，按 Models / Skills / Plugins / Agents 切换资源类型，并在目录中搜索或筛选状态。
2. 用户选择资源后在右侧查看身份、作用域、使用关系、版本和审计；Models 默认显示路由、预算和健康，Plugins 默认显示权限和工具。
3. 编辑 Model、Skill 或 Agent 时进入显式 Edit 状态；Save 创建新的配置版本并记录操作者、差异和时间。

4. 安装 Plugin 前预览权限；连接成功后仍需在 Agent 的 Capability 配置中显式授予，禁止自动继承。
5. 新建 Skill/Agent 先创建 Draft；通过验证并完成权限检查后才能 Publish/Enable。
6. 暂停 Agent 只阻止新 Run；运行中任务继续遵循各自的 Run policy，并在 UI 中明确提示。

10. 状态模型

```
[mermaid]
stateDiagram-v2
    [*] --> Draft
    Draft --> Planning
    Planning --> Running
    Running --> NeedsInput
    Running --> NeedsApproval
    Running --> Paused
    Running --> ReadyReview
    Running --> Failed
    Running --> Disconnected
    NeedsInput --> Running
    NeedsApproval --> Running
    Paused --> Running
    Disconnected --> Running: Reattach
    ReadyReview --> Running: Request changes
    ReadyReview --> Completed: Approve / Deliver
    Failed --> Running: Retry from checkpoint
    Failed --> Cancelled
    Running --> Cancelled
```

状态要求：

- 状态必须来自权威 Run，不可仅由本地动画推断。
- 每次更新包含 revision、timestamp、source、execution_id。
- Thinking 只能作为 Running 的子阶段，不得作为顶层状态。
- Stop 与 Pause 分离；Stop 需要说明是否可恢复。
- Disconnected 表示 UI 与执行失联，不等同于 Failed。

11. 权限与审批

11.1 审批卡必备字段

- Action：即将执行的动作。

- Target : 目标系统、文件、分支、账号或人员。
- Data : 将读取、发送或修改的数据。
- Reason : 为什么当前任务需要此动作。
- Risk : Low / Medium / High , 包含结构化理由。
- Reversibility : 可逆、部分可逆、不可逆。
- Scope : 仅本次、此 Task、此 Project、组织策略。
- Evidence : Diff、Preview、来源或验证结果。

11.2 典型需要审批的动作

- 向外部系统发送消息、评论、表单或文件。
- 修改云端数据、权限、账号和自动化。
- 推送代码、创建/合并 PR、删除或重写历史。
- 提升终端/网络/文件系统权限。
- 跨 Project 使用敏感记忆或客户数据。

11.3 设计约束

- 审批必须位于相关 Canvas 旁边。
- 主按钮使用具体动词，例如“Approve 6 file changes”，不用“Continue”。
- 拒绝后允许补充原因并自动转为 Request changes。
- 不用颜色作为唯一风险信号。

12. 功能需求

12.1 MVP (P0)

- FR-001 : 统一 Project / Task / Thread / Run / Artifact 模型。
- FR-002 : Home、My Work、Projects、Search、Settings 基础导航。
- FR-003 : Work / Code 任务创建与任务内切换。
- FR-004 : 权威运行状态、心跳、Revision 和恢复。

- FR-005 : Work Canvas : Sources、Browser、Document/Artifact。
- FR-006 : Code Canvas : Plan、Changes、Terminal、Browser/Preview、Artifact。
- FR-007 : Local / Worktree / Cloud 环境选择 ; MVP 可先交付 Local + Worktree。
- FR-008 : 上下文审批与结构化 Decision 记录。
- FR-009 : Artifact 版本、审阅、要求修改与导出。
- FR-010 : 通知只针对 Needs input、Needs approval、Ready review、Failed、Disconnected。
- FR-011 : Command deck 支持附件、@ Context、/ Commands、Mode、Model、Autonomy。
- FR-012 : 键盘导航、命令面板和全局搜索。
- FR-013 : Manage 统一资源目录 , 覆盖 Models、Skills、Plugins、Agents 的搜索、状态、详情与作用域。
- FR-014 : Model 路由、Fallback、预算、健康和使用关系管理。
- FR-015 : Skill 指令、工具边界、验证、版本和 Agent 使用关系管理。
- FR-016 : Plugin 安装/更新/禁用、权限预览、工具清单和活动审计。
- FR-017 : Agent 的 Model / Skills / Plugins / Memory / Autonomy 装配、启停和评估。

12.2 P1

- FR-101 : Automations , 支持 Manual / Schedule / Event triggers。
- FR-102 : 企业连接器与最小权限工具选择。
- FR-103 : PR 生命周期、CI、Review comment 修复和 Merge policy。
- FR-104 : 多 Agent 并行、Subagent 可视化和资源预算。
- FR-105 : 跨 Project Memory search 与来源追溯。
- FR-106 : Remote devbox / SSH 与云环境。
- FR-107 : 团队共享模板、Skills、Plugins 与 Agent 配置的发布审批。
- FR-108 : 离线只读与跨设备恢复。

12.3 P2

- FR-201 : 自定义 Canvas 扩展。
- FR-202 : 跨设备 Remote control。
- FR-203 : 组织级 My Work 与 Reviewer 队列。
- FR-204 : 成本、质量、自动化和治理分析。
- FR-205 : Agent / Workflow Marketplace。

13. 非功能需求

- NFR-001 : 冷启动到可操作 Shell , $p95 \leq 3$ 秒 (不包含首次模型/连接器认证) 。
- NFR-002 : Task 切换后 300ms 内显示 Skeleton , $p95$ 1 秒内显示可读历史。
- NFR-003 : 断线重连后 $p95$ 3 秒内恢复权威状态或明确显示 Disconnected。
- NFR-004 : Crash-free desktop sessions $\geq 99.5\%$ 。
- NFR-005 : 任务执行与 UI 状态更新支持幂等和单调 Revision。
- NFR-006 : 1440×1024 为主设计基准 ; 1280×800 完整可用 ; 侧栏可折叠。
- NFR-007 : WCAG 2.2 AA ; 键盘完成所有主流程 ; 状态不只依赖颜色。
- NFR-008 : 中英文 UI ; 日期、数字、快捷键和术语本地化。
- NFR-009 : 默认遵循 `.gitignore` 与企业排除规则 ; 索引范围可见。
- NFR-010 : 审计日志记录 Agent、工具、输入、输出、理由、延迟、权限和结果。

14. 成功指标

北极星指标

每周“可验证完成任务数”：任务具有明确目标、权威完成状态、至少一个可检查 Artifact/变更以及完整 Decision/验证记录。

体验指标

- 首次创建任务成功率。
- 从进入应用到找到“需要我处理”的任务用时。

- Needs approval 到 Decision 的中位时间。
- Ready to review 到交付/合并的中位时间。
- 任务状态误判报告率。
- 中断后 Reattach 成功率与重复执行率。
- Artifact 首次审阅通过率、平均修改轮次。
- Work → Code / Code → Work 切换后上下文保留率。
- 每个完成任务的跨应用切换次数。

工程指标

- Crash-free sessions、ANR、内存峰值。
- Event revision gap、重复事件、状态回滚次数。
- Worktree 创建/清理/Hand-off 成功率。
- Terminal/Browser/Preview 启动成功率。
- Diff / Artifact 渲染 p95。

15. 埋点事件

- task_created、task_mode_changed、run_started、run_state_changed。
- approval_requested、approval_decided、approval_expired。
- artifact_created、artifact_opened、artifact_reviewed、artifact_exported。
- environment_selected、worktree_created、handoff_started、handoff_completed。
- reconnect_started、reattach_succeeded、recovery_forked、duplicate_run_blocked。
- canvas_tab_opened、task_switched、my_work_filter_changed。
- permission_prompt_shown、permission_denied、permission_scope_saved。
- managed_resource_created、managed_resource_updated、managed_resource_published。
- model_route_changed、model_budget_alerted、skill_validation_completed。

- plugin_installed、plugin_permission_changed、plugin_disabled、agent_enabled、agent_paused。

所有主观分类（风险、任务状态原因、下一动作建议）必须记录 Agent 结构化 Tool-call 结果和 rationale，不能由文本关键词规则直接决定。

16. 发布阶段

Alpha

- 现有桌面应用用户，Local + Worktree。
- Work：Research → Document Artifact。
- Code：Plan → Changes → Terminal → Review。
- 验证状态与恢复正确性优先于功能数量。

Beta

- My Work、Automations、连接器、PR 生命周期、多 Agent。
- 企业管理员策略和审计导出。
- Windows/macOS 同步体验。

GA

- Cloud / Remote environments、组织级治理、SLA、跨设备恢复。

17. 风险与缓解

- 模式混淆：用任务目标自动建议，Mode 可见但不强迫理解内部工具。
- 信息过载：默认只显示当前阶段、下一决策和最终产物；日志按需展开。
- Worktree 误操作：环境永久可见、动作预览、不可用原因解释、Handoff 有步骤。
- 状态漂移：服务端权威 Revision、断线状态独立、恢复幂等。
- 隐私和外部副作用：最小权限、数据目的地可见、审批与审计。
- 性能：索引预算、虚拟列表、任务状态增量加载、Canvas 延迟挂载。
- 当前前端单体复杂度：先拆分 Shell/Task/Canvas/Decision，再扩展新 UX。

18. 验收标准

1. 同一 Project 内可创建 Work 与 Code Task，并在 My Work 中统一呈现。
2. 任一任务 Header 可回答：模式、环境、分支/作用域、状态、最后更新、执行 ID。
3. Running、Needs input、Needs approval、Paused、Ready review、Failed、Disconnected 可区分。
4. 通用任务可从来源生成可审阅 Artifact，外部写操作必须审批。
5. 编程任务可在 Worktree 中修改、测试、Preview、审阅 Diff，并安全 Handoff/PR。
6. 审批卡显示动作、目标、数据、风险、可逆性、范围和证据。
7. 重启客户端后可 Reattach 活跃 Run，不重复执行。
8. 所有主流程可用键盘完成，1280×800 不遮挡关键 CTA。
9. 中英文切换后布局无截断，术语一致。
10. 埋点和审计能重建一次任务从意图到交付的完整时间线。
11. Models、Skills、Plugins、Agents 在同一 Manage 工作台可搜索、筛选、查看详情和使用关系。
12. 用户可编辑并保存 Model/Skill/Agent 配置、安装或禁用 Plugin、暂停或启用 Agent，且每个动作产生可见状态反馈。
13. 1100×800 紧凑桌面窗口仍保留资源目录、详情、状态和主动作，无横向溢出。

19. 与现有仓库的实现映射

可复用表面：

- `features/chat/ChatPanel.tsx` → Timeline / Command deck。
- `features/workspace/WorkspaceDock.tsx` → Adaptive Canvas 容器。
- `features/board/BoardPanel.tsx` → My Work / Runs 任务视图的部分能力。
- `features/sandbox/SandboxPanel.tsx` → Terminal / Preview 环境能力。
- `features/runtime/RuntimeConfigPanel.tsx` → Environment 与连接设置。
- 已有 task、artifact、plan、event 类型 → Run Timeline 与 Artifact 生命周期。

实施前置：

- 将 `App.tsx` 中 Shell、Navigation、Task selection、Run state、Workspace Canvas、Approval state 拆成独立模块。
- 建立单一 `TaskViewModel` 和权威 `RunState` 适配层，避免每个面板分别推断状态。
- 明确 Workspace / Conversation / Task / Run 的迁移和兼容策略。

UI/UX specification

版本：v1.1 基准画布：1440×1024 最低支持：1280×800 主题：Dark first，Light theme 进入 P1

1. 设计原则

1.1 先回答状态，再展示过程

用户进入任何任务后，应在 3 秒内回答：它是否在运行、是否需要我、在哪里执行、上次更新是什么时候。Timeline 不能用动画代替权威状态。

1.2 工作表面优先于聊天

聊天用于表达意图、解释和调整。Document、Data、Browser、Changes、Terminal、Preview 和 Artifact 才是主要协作表面。

1.3 一个任务，一个当前决策

同一时刻只突出一个当前必须处理的 Decision。其他审批进入队列，避免页面同时出现多个同权重 CTA。

1.4 环境永不隐身

Code Task 的 Local / Worktree / Cloud、Branch、Checkout 和 Preview 地址永久可见。Work Task 的 Project、Privacy scope 和外部数据范围永久可见。

1.5 高级能力渐进呈现

非开发者默认只看到目标、来源、进度、产物和审批；开发者可以展开终端、工具事件、Execution ID 和资源数据。

2. 视觉基础

2.1 色彩 Token

- `--desktop-bg: #080C12`
- `--desktop-panel: #0F141D`
- `--desktop-panel-2: #151A24`
- `--desktop-border: #242B36`
- `--desktop-border-soft: #1B222E`
- `--desktop-text: #E7EDF6`
- `--desktop-muted: #9AA5B5`
- `--desktop-faint: #687386`
- `--desktop-cyan: #38D6FF`
- `--desktop-green: #35D399`
- `--desktop-amber: #F0B35A`
- `--desktop-red: #FF6978`
- `--mode-work: #7C6CF2` (仅用于模式识别 , 禁止渐变)

2.2 排版

- UI : Inter、SF Pro、Segoe UI 或系统 Sans。
- Code/Terminal : SFMono-Regular、Menlo、Consolas。
- Body : 14px / 20px。
- Secondary : 12px / 16px。
- Page title : 18px / 24px , 600。
- Section title : 14px / 20px , 600。
- 不使用全大写长标题 ; 状态可使用短 Label。

2.3 间距与圆角

- 基础间距：4 / 8 / 12 / 16 / 24 / 32。
- 控件高度：Compact 28，Default 32，Primary 36。
- Composer 最小高度：84；展开上限：220。
- Panel radius：8px；Pill 仅用于 Status / Mode / Filter。
- Shadow 仅用于浮层、菜单、Modal，不用于普通面板。

2.4 图标

- 使用一致的 Outline 图标库。
- 16px 为常规、14px 为密集列表、20px 为一级导航。
- 禁止 Emoji、手绘 SVG 和仅靠颜色表达状态。

3. 桌面 Shell

3.1 左侧 Global Rail

宽度：220–232px；折叠 48px。

结构：

1. Window / Product area
2. Home
3. My Work
4. Manage
5. Automations
6. Search
7. Projects / Tasks
8. Notifications
9. Settings / Account

规则：

- 不把 Memory、Artifacts、Runtime 同时作为一级全局导航；它们分别属于 Search、My Work/Task 和 Settings/Environment。
- Models、Skills、Plugins、Agents 归入一个 Manage 导航项，在工作台内部切换，避免四个管理入口稀释任务主导航。
- Project 展开后显示 Task，不直接复制所有 Canvas Tab。
- Running、Needs input、Ready review 使用图标 + 文本/Tooltip，不能只显示彩色点。

3.2 Task Header

高度：48–56px。

左侧：Task title、Project。中部：Mode、Environment/Privacy、Branch/Scope。右侧：State、Elapsed、Usage、Pause/Stop、More。

环境信息可压缩但不可消失。1280 宽度下，Elapsed 与 Usage 进入 More，Mode/Environment 保留。

3.3 Command Deck

位于任务底部，默认单行 + 工具行。

第一优先级：输入与 Send/Steer。第二优先级：Work/Code、Context、Attach。第三优先级：Model、Autonomy、Tools。

快捷方式：

- `Cmd/Ctrl + K`：命令面板。
- `Cmd/Ctrl + P`：文件/Artifact 快速打开。
- `Cmd/Ctrl + Shift + M`：Work / Code 切换。
- `Cmd/Ctrl + Shift + B`：Browser。
- `Ctrl + Backtick`：Terminal。
- `/`：命令；`@`：上下文；`#`：Issue/对象引用。

4. 自适应 Canvas

4.1 Work 模式 Tab

- Sources : 来源列表、覆盖度、引用状态。
- Browser : 研究与网页交互。
- Document : 文档/演示/表格编辑或预览。
- Data : 表格、图表、Notebook/SQL 结果。
- Artifact : 最终产物、版本、导出和分享。

4.2 Code 模式 Tab

- Plan : 可编辑计划和验收条件。
- Browser : 本地 Preview、网页 QA。
- Changes : 文件列表、Diff、Comment、Stage/Revert。
- Terminal : 任务作用域终端与验证结果。
- Artifact : 构建物、测试报告、PR packet、文档。

4.3 分屏

- 仅在任务需要同时比较两个表面时出现，例如 Preview + Changes。
- 默认 50/50，可拖拽到 35/65。
- 同一屏不超过两个主 Canvas；第三表面使用 Tab。
- Decision rail 宽度 280–320px，可折叠。

5. Timeline

展示：

- User intent
- Editable plan
- Current phase
- Key actions

- Validation results
- Decisions
- Final summary

默认隐藏：

- 完整工具参数
- 重复进度消息
- 原始事件 JSON
- 模型内部推理

Tool activity 以“Search → Edit files → Run tests → Type check”紧凑呈现；点击后在 Drawer 展开日志。

6. 关键组件

6.1 Task row

字段：Status icon、Task title、Mode、Project/Agent、Last update、Action hint。

优先排序：Needs approval > Needs input > Failed > Ready review > Running > Scheduled > Completed。

6.2 Run status

- Running：绿色 + 动词阶段，例如“Running tests”。
- Paused：Amber + “Paused by Alex”。
- Needs input：Amber + 问题摘要。
- Needs approval：Amber border + 具体动作。
- Ready to review：绿色 + Artifact/Changes 数量。
- Failed：红色 + 最近成功 Checkpoint。
- Disconnected：灰色/红色 + Last heartbeat + Reconnect。

6.3 Decision card

结构：Title → What will happen → Risk/Scope → Evidence → Primary/Secondary action。

高风险动作必须在按钮中体现对象，例如：

- “Push 2 commits”
- “Send report to Slack #leadership”
- “Approve 6 file changes”

6.4 Artifact viewer

- 左侧 TOC / Sheet / File list。
- 主区域内容。
- 顶部 Version、Status、Source coverage。
- 底部 Review、Ask for changes、Export/Share。
- Artifact 支持 Draft、Ready、Approved、Delivered、Superseded。

6.5 Environment switcher

选项：Local、Worktree、Cloud。

每项描述结果而非技术名词：

- Local：直接使用当前目录，可能影响你的工作副本。
- Worktree：为此任务创建隔离副本。
- Cloud：在远程隔离环境运行，本机可关闭。

切换前预览未提交变更、Branch、依赖设置和 Handoff 步骤。

6.6 Management category rail

- 位于 Global rail 与 Resource catalog 之间，桌面宽度 140–166px。
- 固定四类：Models、Skills、Plugins、Agents；每项显示 Icon、名称、用途摘要和数量。
- 选中态使用 2px Cyan 左标记、深色填充和文字高亮；不只依赖颜色，必须保留名称与语义图标。
- 底部固定 Governance note，提示配置版本化与可审计。

6.7 Resource catalog

- 宽度 244–282px；结构为标题/Add、Search、All/Active/Attention、排序、资源列表。
- 每行显示统一字段：Icon、Name、Description、Version/meta、Status；状态使用 Dot + Label。
- 列表支持搜索、状态过滤、选中态和空结果；不可把不同资源的专用字段强塞入同一列。

6.8 Resource detail

- 顶部 48px：Breadcrumb、Scope、Copy link、当前主动作。
- Identity：类型、名称、描述、Status、Tags、Owner/Provider。
- 公共 Tab 模型：Overview + 3 个专用 Tab；Tab 数量固定为 4，避免深层设置树。
- Overview 先显示 3–4 个关键指标，再显示依赖/路由/权限关系。
- Edit 状态必须显式进入，Save 后返回只读并显示成功反馈；配置变更提示“创建新的审计版本”。

6.9 类型专用内容

- Models：Overview、Configuration、Usage、Audit；突出 Success、Latency、Spend、Fallback chain 和 Agents using。
- Skills：Overview、Instructions、Validation、Versions；突出 Skill contract、Allowed tools、Validation pass 和版本。
- Plugins：Overview、Permissions、Tools、Activity；突出 Publisher、Install/Update/Disable、数据权限和工具清单。
- Agents：Overview、Behavior、Capabilities、Evaluations；突出 Model routing、Memory boundary、Autonomy、Skills 与 Plugin dependencies。

7. 屏幕清单

S01 Home / Today

- 今日需要处理、正在运行、可恢复任务。
- 最近 Projects 和推荐自动化。

- 主 CTA : New task。

S02 My Work

- Needs input、Needs approval、Running、Ready review。
- Work/Code 混合队列。
- 过滤 Project、Mode、Owner、Status。

S03 New Task

- **S03A Describe task** : Goal、Project、Work/Code、Context、Environment/Privacy ; 明确提示“Agent 尚未执行”。
- **S03B Agent planning** : 展示 Agent 正在理解目标、读取获准上下文、选择执行路径和准备 Review packet ; Authority 固定为 Plan only。
- **S03C Human plan review** : 逐步展示步骤、每步产物和时间 ; 支持编辑、禁用、增补、Ask agent to revise。
- 右侧 Run Preview 固定显示预计耗时、Usage、上下文、执行边界和授权摘要。
- 主 CTA 为 Approve & start task ; 批准前不创建 Run、不运行工具、不修改文件。
- 高级设置默认折叠。

S04 Quick Chat

- 无正式 Run/环境。
- CTA : Create task from this chat。

S05 Work Task - Running

- Sources + Timeline + Draft Artifact。
- 显示来源覆盖与当前阶段。

S06 Work Task - Needs Approval

- Browser/Document 旁显示外部写操作审批。

S07 Work Task - Ready Review

- Artifact 为主 , Review / Ask changes / Export。

S08 Code Task - Planning

- Plan、验收标准、Environment/Branch。
- CTA : Approve plan and run。

S09 Code Task - Running

- Timeline + Changes/Terminal/Preview。
- Pause/Stop 永久可见。

S10 Code Task - Needs Approval

- Diff/Preview + Risk + Affected scope。

S11 Code Task - Ready Review

- Files changed、Tests、Findings、Commit/PR actions。

S12 Recover / Reconnect

- Last heartbeat、Run ID、Environment、Reattach、Inspect、Fork recovery。

S13 Automations

- Name、Project、Trigger、Environment、Last run、Next run、Permission profile。

S14 Search / Memory

- 跨 Task、Thread、Artifact、Source、Entity。
- 显示来源、Project、时间和可用范围。

S15 Settings / Governance

- Browser、Terminal、组织 Permissions、Memory、Retention、Usage ; 资源管理已迁移到 Manage。

S16 Manage / Models

- 目录显示 Provider、上下文、健康、预算告警与 Offline。
- Detail 显示 Runtime chain、Fallback、Latency、Spend、Success 和使用它的 Agents。
- 主动作 : Edit config ; Configuration 保存时创建审计版本。

S17 Manage / Skills

- 目录显示 Published / Draft / Test failed、版本和工具数量。
- Detail 显示 Skill contract、Instructions、Allowed tools、Validation 与 Versions。
- 主动作：Run validation；Create skill 先进入 Draft。

S18 Manage / Plugins

- 目录显示 Connected / Update available / Not installed。
- Detail 显示 Permissions、Tools、Activity、Publisher 和 Agents using。
- 主动作根据状态为 Install plugin / Update plugin / Disable plugin。

S19 Manage / Agents

- 目录显示 Active / Paused、默认 Model 和 Skill 数量。
- Detail 显示 Model routing、Memory boundary、Autonomy、Skills、Plugin dependencies 和 Evaluations。
- 主动作：Pause agent / Enable agent；Create agent 先创建 governed draft。

8. 空、错、慢状态**空状态**

- 不展示功能清单；只解释当前页面的任务与一个主动作。
- My Work 空：说明“没有等待你的任务”，可查看 Running 或 New task。
- Project 空：New task、Add sources、Connect repository 三选一，按 Project 类型推荐。

加载状态

- Shell 先出现，Task Header 与权威状态优先加载。
- Canvas 可延迟；显示具体“Loading diff / Reconnecting terminal”。
- 不用无限旋转动画掩盖失联。

失败状态

- 显示发生位置、最近成功 Checkpoint、是否有副作用、推荐恢复动作。

- Retry 与 Fork recovery 分离。
- Error detail 可复制但默认折叠。

9. 响应式与窗口尺寸

≥ 1440px

Global rail + Timeline/Queue + Canvas + 可选 Decision rail。

Manage 使用 Global rail 150px + Category rail 166px + Catalog 282px + Detail 自适应。

1280–1439px

Global rail 可折叠；Timeline 和 Canvas 二选一主宽，Decision 使用 Drawer。

1024–1279px (非主支持)

单主 Canvas；Timeline 变为可切换 Tab；Command deck 简化 Model/Autonomy 到菜单。

不得在小窗口中把关键状态、Environment、Pause/Stop 或审批 CTA 隐藏到无法发现的位置。

Manage 在 1100px 宽度下压缩为 Category rail 140px + Catalog 244px，Detail 的 Owner/Provider 侧栏隐藏，关系卡改为单列；Breadcrumb、状态与主动作继续可见。

10. 可访问性

- WCAG 2.2 AA 对比度。
- 所有交互可用键盘完成，Focus ring 不被移除。
- Status icon 有可读 Label；颜色不是唯一通道。
- Canvas Tab、Task row、Decision 使用正确语义角色和 `aria-live` 策略。
- 流式更新不持续抢占屏幕阅读器；只播报阶段和需要用户行动的状态。
- 动画支持 Reduce Motion。
- Diff 支持新增/删除符号与文本，不只用红绿底色。
- 表格、文档和图表提供替代文本或数据表。

11. 文案规范

- 用结果动词：Review brief、Approve changes、Reconnect task。

- 避免含糊词：Continue、Proceed、Working、Thinking。
- 状态包含主体：Paused by you、Waiting for Agent、Needs your input。
- 环境术语首次出现附解释；后续保留短 Label。
- 风险说明写影响，不写恐吓文案。
- 中文界面保留必要技术词：Worktree、Branch、Commit、PR，但提供 Tooltip。

12. 三套视觉方向

方向 1：Adaptive Task Canvas

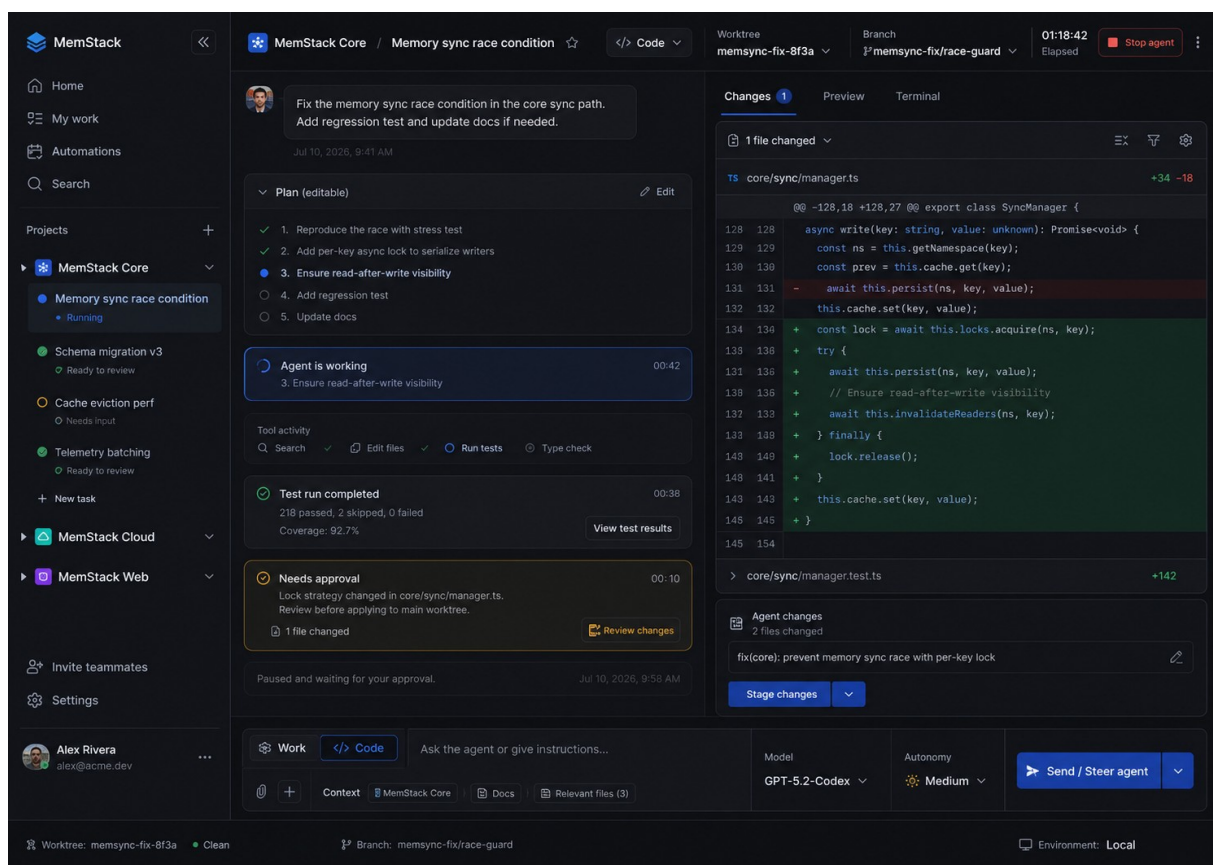


Figure: Adaptive Task Canvas

特点：Task 为核心；Timeline 与 Changes 并列；最适合高频交互和当前客户端渐进迁移。

方向 2：My Work Mission Control

评审结论：已选为桌面客户端主方向。

The screenshot displays the MemStack desktop agent client interface. The top bar shows the project 'Acme Product', privacy 'Acme internal', elapsed time '02:47:18', and usage '\$1.42' (28.3K tokens). A 'Pause agent' button is visible.

The left sidebar contains navigation links: Home, My work (selected), Automations, Search, and Projects. The bottom of the sidebar shows the user profile 'Alex Rivera' from 'Acme Corp'.

The main area is titled 'My work' and shows a queue of tasks:

- Needs your input (1):** 'Q3 product strategy brief' (Work • Agent Atlas, 10:24 AM, NEEDS INPUT).
- Running (2):**
 - 'Fix flaky test in data-pipeline' (Code • Agent Dev, 18m, RUNNING).
 - 'Build cohort retention model' (Work • Agent Sage, 32m, RUNNING).
- Ready to review (3):**
 - 'Q2 results deep dive' (Work • Agent Atlas, Yesterday, READY).
 - 'Refactor auth middleware' (Code • Agent Dev, Yesterday, READY).
 - 'Market sizing update' (Work • Agent Sage, Jul 9, READY).

A 'Show completed' link is also present.

The right panel provides a detailed view of the 'Q3 product strategy brief' task:

- Task Details:** 'Q3 product strategy brief' (Agent Atlas • Work mode, NEEDS INPUT). Started: Jul 10, 2026 8:12 AM. Due: Today 12:00 PM. Description: 'Create a Q3 product strategy brief for Acme's leadership team using company memory, market research, spreadsheets, and recent team discussions.'
- Progress:** 86% Completed. Steps: Understand objective (checked), Collect & analyze (checked), Synthesize brief (checked), Awaiting your review (active).
- Sources used (28):** MemStack memory (12), Spreadsheets (6), Documents (5), Web research (3), Team discussions (2). An 'Open sources' link is provided.
- Blockers (1):** 'Confirm strategic priorities for Q3 focus areas.' A 'Provide input' button is available.
- Draft brief (v1.0):** Includes an 'Overview' section with a table of contents (Executive summary, Market context, Strategic priorities, Go-to-market plan, Financial outlook, Risks & mitigations, Appendix) and an 'Executive summary' section. The summary states: 'Q3 is about accelerating our platform adoption and expanding enterprise footprint. We will double down on three priorities: reliability, developer experience, and ecosystem growth.' An 'At a glance' box lists: 3 strategic priorities, 5 key initiatives, \$2.3M projected impact, and 8 risks identified.
- Actions:** 'Review brief', 'Ask for changes', and 'Open sources' buttons.
- Steer Atlas:** A section for sending follow-up requests or requesting changes.
- Footer:** A status bar indicating 'Atlas has access to Acme internal data. It won't use external data without permission. [Learn more](#)'.

Figure: My Work Mission Control

特点：队列和 Artifact 为核心；最适合跨项目监督、知识工作和 Reviewer。

方向 3 : Shared Workspace Deck

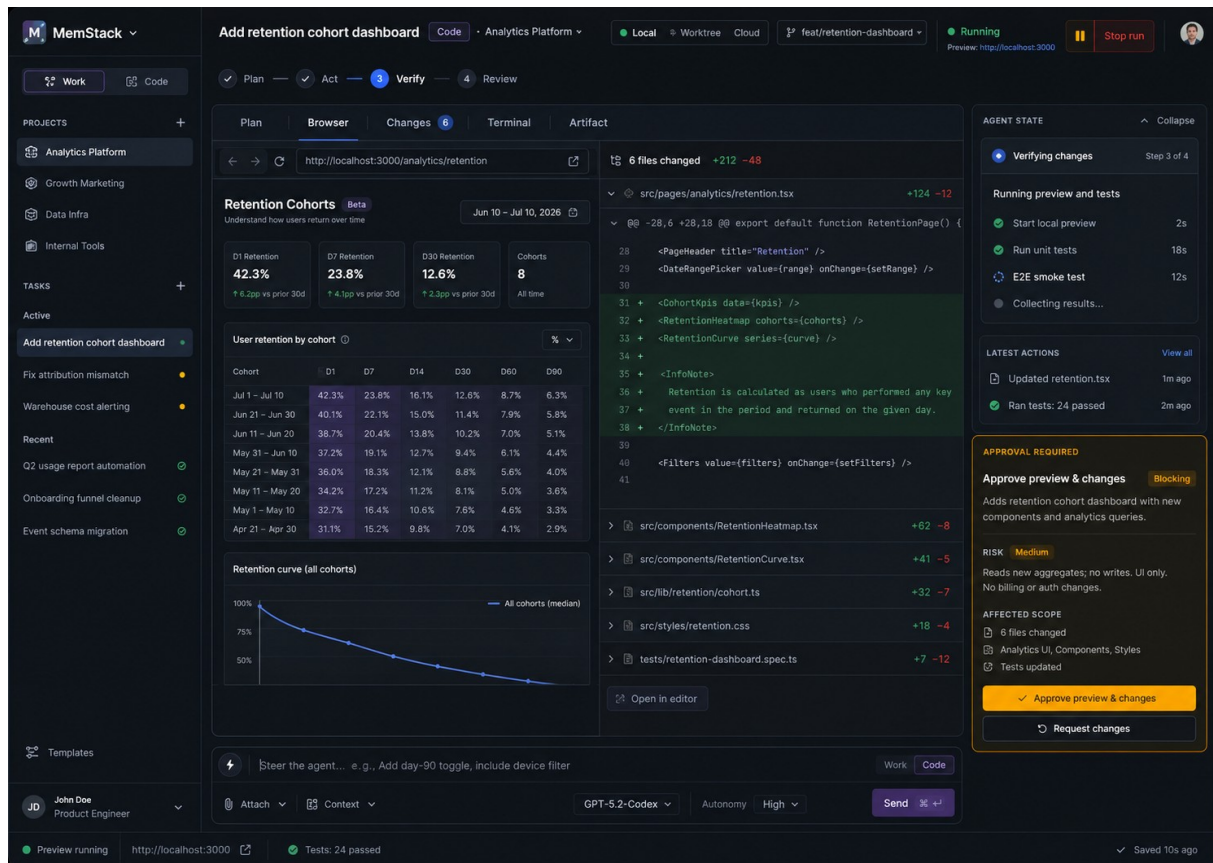


Figure: Shared Workspace Deck

特点：Canvas 为核心；Preview、Changes、Terminal 和 Artifact 协同最强，适合产品工程混合任务。

13. 视觉方向评审标准

- 通用用户是否能在 10 秒内找到下一步。
- 开发者是否能确认 Environment、Branch 和变更范围。
- 是否能区分 Task、Thread 和 Run。
- 是否优先展示 Artifact / Changes，而不是日志。
- 审批是否与证据同屏。
- 1280×800 是否仍保留核心 CTA。
- 是否可以复用现有组件并渐进迁移。

已依据主方向完成可交互高保真原型，覆盖 Work / Code 的 Running、Needs input、Ready review、审批、来源、代码变更、终端和 Steer，以及 S16-S19 的 Models、Skills、Plugins、Agents 统一管理工作台。后续生产实现继续按 S01-S19 屏幕清单补齐完整路由、离线恢复与真实数据接入。